

SoK - Network Intrusion Detection on FPGA

Laurens Le Jeune^{1,2}, Arish Sateesan¹, Md Masoom Rabbani¹, Toon Goedemé²,
Jo Vliegen¹, and Nele Mentens^{1,3}

¹ ES&S - imec-COSIC, ESAT, KU Leuven, Belgium

² EAVISE - PSI, ESAT, KU Leuven, Belgium

³ LIACS, Leiden University, The Netherlands

firstname.lastname@kuleuven.be

Abstract. The amount of Internet traffic is ever increasing. With a well maintained network infrastructure, people find their way to Internet forums, video streaming services, social media and webshops on a day-to-day basis. With the growth of the online world, criminal activities have also spread out to the Internet. Security researchers and system administrators develop and maintain infrastructures to control these possible threats. This work focuses on one aspect of network security: intrusion detection. An Intrusion Detection System (IDS) is only one of the many components in the security engineer's toolbox. An IDS is a passive component that tries to detect malicious activities. With the increase of Internet traffic and bandwidth, the detection speed of IDSs needs to be improved accordingly. This work focuses on how Field-programmable Gate Arrays (FPGA) are used as hardware accelerators to assist the IDS in keeping up with high network speed. We give an overview of three approaches: Intrusion detection based on machine learning, pattern matching, and large flow detection. This work is concluded with a comparison between the three approaches on the most relevant metrics.

Keywords: Network intrusion detection, Deep Learning, FPGA

1 Introduction

Stating that the amount of Internet traffic is increasing, is like kicking at an open door. The global COVID-19 pandemic contributed in raising the total amount of network traffic to unprecedented volumes. This was mainly due to the lock downs that were installed in various parts of the world, boosting video conferencing and on-line shopping. It should not come as a surprise that malicious individuals or groupings saw this as a lucrative evolution. Even without this evolution, network security is becoming increasingly important.

The simplest way to describe the Internet is calling it a network of networks. Fig. 1. helps with introducing a number of concepts or components that are typically used in organising network security. The local network (e.g. the office network or a home network) is referred to as the Local Area Network (LAN). All the devices in the network can communicate with each other. When the LAN

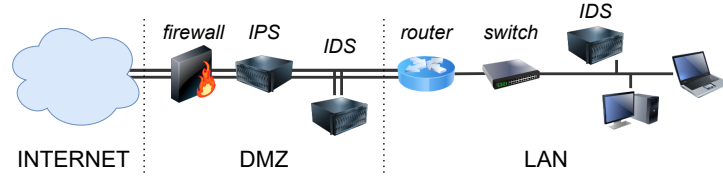


Fig. 1. General network setup

is hooked up to the Internet, a number of components are added in between. Generally these devices reside in a so-called demilitarised zone (DMZ).

The device, in the DMZ, that is positioned the closest to the Internet is the traditional firewall. This firewall uses a set of rules which it traverses sequentially. Depending on matches or mismatches to certain rules, incoming (or outgoing) traffic is allowed through or is denied passage. The firewall typically operates standalone and provides a first layer of network protection in an active way and focuses primarily in a single direction: from the Internet to the LAN. The actions taken by a firewall are typically also logged on the system.

An Intrusion Detection System (IDS) has a more passive role. Network traffic that is allowed by the firewall is examined by the IDS, as shown in Fig. 1. The IDS searches for signatures of network attacks. When such symptoms are detected, the system administrators can be notified. The logs of the passive IDS can be used to strengthen the active firewall. By using an IDS, additional defenses can be set up against threats like access of unauthorised clients or servers, Trojan horses, viruses, policy violations, SQL injections or Distributed Denial of Service (DDoS) attacks [35]. An IDS could also be placed in the local network to detect potential malicious activities from within the private network. In this work, we concentrate on network-based intrusion detection systems (NIDSs), which monitor the traffic in the network, as opposed to host-based intrusion detection systems (HIDSs) that monitor a specific host. In fact, the IDS in Fig. 1 is an NIDS. Fig. 2 introduces this classification.

A challenge that comes with the design of an NIDS is that it should detect potential attacks as fast as possible in order to reduce the window of opportunity for an attack to be exploited. Therefore, this work focuses on NIDSs that are supported by dedicated hardware accelerators implemented on a Field-Programmable Gate Array (FPGA). Although with each new FPGA generation, larger designs can be supported, it is not always trivial to fit an NIDS in the configurable resources of one FPGA. When the FPGA needs additional external components, this has a detrimental effect on the processing speed. Moreover, the larger the utilized configurable resources, the longer it takes to run the toolchain for each new design. When frequent updates of the NIDS are needed, the redesign and reconfiguration delay plays an important role.

Although many methods can be used, this work approaches the IDS from three different angles based on the technique that is used to make a decision: pattern matching, large flow detection, and machine learning (see Fig. 2). While re-

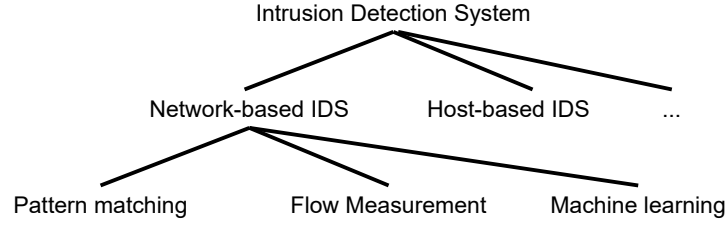


Fig. 2. IDS classification

lated work typically concentrates on only one technique, the contribution of this paper is that it makes a qualitative comparison between the three approaches. This allows practitioners to make a decision on which method to use based on the specific application scenario. The three different approaches are discussed in Sect. 2, Sect. 3, and Sect. 4. Subsequently, a number of relevant properties are compared in Sect. 5 and the work is concluded in Sect. 6.

2 Pattern Matching-based Network Intrusion Detection

When a network frame has been captured and has passed the firewall, it is inspected by the NIDS. An NIDS that follows the pattern matching approach then searches for signatures in the frame that belong to known network attacks in an extensive database. This database is generated (and maintained) by security engineers and contains a long list of *rules*. The number of rules can be as high as hundreds of thousands [54]. An example of a rule could be, for example, that an alarm should be triggered if network frames arrive, destined for port number 80 (which is the default port for an HTTP server). This rule could have been added because the policy states that only HTTPS traffic is allowed (which uses port 443).

Because of the large amount of rules that need to be verified, it becomes infeasible to linearly go over all of them. The main software suite that serves IDS or IDPS is Snort⁴. Other available systems include Zeek⁵ (formerly known as Bro) and Suricata⁶. In Snort, the comparison with the active rule-set is done through string matching and/or regular expression (RegEx) string matching. Wang et al. published Hyperscan [70] which is a software-library to facilitate both. FPGA architectures for pattern matching-based NIDS are based on three different approaches: deterministic solutions, non-deterministic finite automata, and probabilistic architectures.

Deterministic solutions for pattern matching: Zhao et al. present the IDS/IPS Pigasus in [78]. They devise Pigasus as an FPGA-first implementation where the FPGA is not a co-processor to assist the main processor (which

⁴ <https://www.snort.org/>

⁵ <https://zeek.org/>

⁶ <https://suricata.io/>

is doing the heavy lifting), but where the FPGA executes the majority of the processing. Pigasus can support a 100 Gbps network speed and consumes $38\times$ less power than a CPU-first implementation. In their work, the authors present an FPGA implementation on an Intel FPGA.

Kang et al. presented in 2019 an FPGA-based implementation of a rule-based IDS [29]. This work is based on a ‘whitelist’ that can hold up to 256 entries. The ruleset can be updated through an on-chip processor. The latency is $0.032\mu\text{s}$ which is mentioned to allow detection even before the complete packet has arrived.

Non-deterministic finite automata for pattern matching: Software IDSs heavily lean on the power of RegExs to provide flexible and high-performance searches. Unfortunately, checking RegExs on FPGAs is not straightforward. Luinaud et al. overcome the implementation challenge by using the reconfigurable nature of FPGAs [42]. This is achieved by using Non-deterministic Finite Automata (NFA), which is an approach to implement Finite State Machines (FSMs). In contrast with DFA (Deterministic Finite Automata), NFA can result in a different output state for a given input state and a set of inputs. Luinaud et al. use an intermediate layer on top of the classical reconfigurable fabric. Through this they are able to implement NFAs.

In [14], Češka et al. implement resource-efficient RegEx matching by approximately reducing NFAs based on the probabilistic distance between an automaton and its reduced counterpart. They propose heuristics to approximate this reduction, using a probabilistic model of the network traffic. Using this approach allows for significantly reducing automata while limiting the resulting error. As a result, they are able to accelerate the detection of Snort rules for 100Gbps. Moreover, in [15], the authors implement multiple stages of over-approximating NFAs to push the possible throughput even further. Consider the traffic profile of the local networking environment they once again reducing the NFAs to limit the impact on accuracy. With their resulting multistage hardware accelerators, they are able to process traffic of up to 400Gbps for some Snort modules.

Probabilistic architectures for pattern matching: The most commonly used architectures for implementing of pattern matching algorithms on hardware are Bloom filters [12], content addressable memories (CAM), and ternary content addressable memories (TCAM) [2]. Even though CAMs and TCAMs are mighty accurate, disadvantages such as high cost per bit, storage inefficiency, limited scalability to long input keys, and large memory overhead put CAMs and TCAMs on the back foot [2]. In bloom filters, the length of the string has no effect on the memory requirements and the query time is not affected by the number of strings in the database. These advantages give a significant boost to bloom filters when it comes to hardware. These properties hold the same for other probabilistic lookup architectures such as Cuckoo filters [23] too.

Numerous hardware-focused researches on the implementation of signature-based IDS using deep packet inspection (DPI) and prefix matching schemes employ bloom filters as the main component. Dharmapurikar et al. [19] proposed a DPI system using parallel bloom filters, which employs a set of parallel bloom fil-

ters where each bloom filter contains predefined signature of a particular length. Considering the fact that there could be false positives, an analyzer based on a deterministic string matching algorithm, followed by the set of bloom filters ensures that false positives are eliminated. However, these analyzer units are slow and would affect the throughput when the false positive rates are high. Artan and Chao [9] proposed a chain heuristic method to reduce the false positive rate of Prefix bloom filters (PBF) without additional memory requirements, hence reducing the operation of the analyzer. Prefix bloom filters is the term given to the bloom filters stored with signatures and it used the same set of parallel bloom filters as in [19]. Additionally, this method also helps to detect signatures distributed over multiple packets, which [19] could not detect. A similar architecture using counting bloom filters as a system on chip (SoC) implementation is performed by Harwayne-Gidansky [26]. In this SoC, the operation of the analyzer part is performed by the PowerPC softcore processor.

Work by Dharmapurikar [20] extend the functionality of [19] to longest prefix matching (LPM) by employing additional off-chip hash tables. Based on the match vector from prefix bloom filter outputs, hash table lookups are performed for matching prefixes. However, this method would result in reduced throughput because of the multiple off-chip memory accesses. A slight modification to longest prefix matching using parallel counting bloom filters is put forward by Li [36], which requires less memory but at the cost of reduced speed. Extended bloom filter (EBF) proposed by Song and Lockwood [62] is a modified version of bloom filter for string matching. Each bucket in the bloom filter consists of 3 fields, hit-bit, a counter, and a pointer. Hit-bit is the same as in bloom filters whereas counter field is used to accelerate the matching process and pointer points to the signature storage location in the off-chip memory. In order to limit the false positive rate, either the number of EBF buckets or number of hash functions must be increased, which increases the memory consumption. The pointer field is 11-bits, which also adds to the high memory consumption compared to standard bloom filters. The use of off-chip memory in EBF also decreases the throughput.

Most of the deep packet inspection/string matching approaches based on bloom filters are static and also many of the techniques do suffer from a low throughput. Kefu et al. [30] proposed a counting bloom filter based architecture with two pipeline stages, fast and slow, on FPGA. The fast pipeline stage consisting of parallel counting bloom filter engines which is very much similar to [19], and the slow pipeline consists of a dynamic pattern matching algorithm. The fast pipeline ensures fast dynamic query and a high throughput whereas the slow pipeline validates the suspicious matches from the fast pipeline and at the same time updates the rule sets dynamically at run-time. Dharmapurikar and Lockwood [21] proposed another approach to improve the throughput by minimising the off-chip memory usage. In this work, prefix bloom filters are implemented in on-chip memory and hash tables in off-chip memory. The hash table lookup is performed only when there is a match in the bloom filters. This drastically reduces the off-chip memory access time and hence the throughput is improved. Aggregated bloom filter (AGF) [10] is a completely different ap-

proach than the parallel bloom filters. In AGF, each query is serialized into a queue and multiple queries can be performed simultaneously by one bloom filter, unlike parallel bloom filters where it can perform only one query at a time. Al-Dalky et al.[3] employed the NetFPGA as a hardware line of defense to accelerate the Snort NIDS. A bloom filter is used for matching the Snort rules and the most frequently triggered rules are dynamically offloaded to the NetFPGA. The NetFPGA replaces the Ethernet card and for every new incoming packet, the packet is passed to the slow Snort IDS module only if no matches are found in the NetFPGA. This minimizes the processing burden on the Snort IDS and improves the processing rate.

Even though not entirely specific to intrusion detection, there are numerous works available presenting the improvements of bloom filters for pattern matching applications on FPGA. The work by Wada [69] presents an FPGA implementation using UltraRAMs and rolling hash functions, providing a significant boost in performance. A sublinear time string matching algorithm using bloom filters is presented by Lin et al. [37] which allows skipping characters which are not matching, and proposed a bad-block heuristic to handover verification job to a verification module. An approach to improve the throughput of bloom filters on hardware is presented by Sateesan et al. [56], which showcases a high-throughput implementation of bloom filter having a single memory access and eliminates the disadvantages of hashing for bloom filters. Other than bloom filters, a few researches are available based on cuckoo filters. Works by Hisnawi and Ahmadi [4] which implements deep packet inspection system on CPU using cuckoo filters by storing the rule signatures on the filter, and the work by Ho [27] which proposes a deep packet inspection system on graphics processing units employing a pattern matching scheme on cuckoo filters provides showcases improved performance.

3 Flow Measurement-based Network Intrusion Detection

With the emergence of high-speed networks and devices, flow measurement has become an important research challenge than ever before [22]. Real-time monitoring and storage of enormous amount of data pose the need of huge amount of resources and memory, which makes the implementation practically impossible and even more difficult on hardware. There has been numerous approaches to alleviate the complexities in flow measurements. Sampling based approaches [16,60] were commonly employed in many of the networking systems, which helped in significantly reducing the memory consumption compared to exact measurement approaches. Even though sampling techniques do achieve memory reduction, it has become obsolete with high-speed networks, and recent studies have pointed out the remarkable reduction in accuracy and loss of information caused by high sampling [39]. The proposal of an approximate counting algorithm in 1978 by Robert Morris [43] give rise to an alternate counting technique in database and networking applications. Along with approximate counting, the introduction of probabilistic counting algorithms based on hash

tables [24] helped in significantly reducing the counting overhead while allowing a small error in accuracy. Sketches [17,74,39] in general significantly reduced the memory overhead of measurement and flow record generation tasks. An in-depth study of the role of sketches for traffic measurement is presented by Zhou in [79].

Sketch algorithms [74] and works employing sketches [72] have shown already that sketch based traffic monitoring schemes can deliver high throughput on FPGA. Work by Wellem et al. [72] employed a sketch data structure to extract the traffic features and the sketch-based monitoring part is implemented on a multi-core CPU. The hashing, bloom-filter based lookup and flow tables are implemented on the FPGA. The system is able to process a packet in every clock cycle at 210MHz and is able to deliver a throughput of 53Gbps assuming a minimum packet size of 64 bytes on a Virtex-5 platform. Approximation approaches such as self adaptive (SA) counters [73] on sketches for per-flow measurement proved to be effective reducing the memory footprint, but SA counters seem to be too complex for hardware. A more hardware efficient method to replace exact counting with approximate counting is proposed by Sateesan et al [57]. This work proposes approximate CM ((A-CM) sketch and showed that the memory overhead can be reduced to half for CM sketch implementation and a high throughput can be obtained even without pipelining. With pipelining, A-CM sketch of width $w=2^{16}$ and depth $d=4$ can process a packet in every clock cycle at 454MHz to achieve a throughput of 233Gbps on a Xilinx ultrascale+ device. Schweller et al. [59] proposes a reversible sketch architecture for traffic monitoring and change detection. Sketches in general are not reversible, which means we cannot track down flows having a large change estimate. This paper by Schweller presents reversible sketches with reverse hashing algorithms that provide an efficient way to network traffic monitoring for change detection. The FPGA implementation is able to produce a throughput of 16Gbps on a virtex 2000E FPGA.

An intrusion detection model on FPGA using sketch-based feature extraction modules (FEM) is proposed by Das et al. [18]. The FEM is associated with an anomaly detection mechanism using principal component analysis (PCA). FEM summarizes the network behaviour and the PCA helps for detecting outliers. The FPGA implementations are performed on a Virtex-II device. The FEM implementation shows a throughput of 23.82Gbps at 216MHz with an accuracy of 97.61% for a sketch width of 2048 and the number of hash functions $h=4$. But with increasing depth, the throughput and frequency decreases. Feature extraction modules and traffic classifiers also play a vital role in the network intrusion detection systems. Sketches have been the most hardware-friendly architecture for in-line feature extraction and classification applications. Similar to the work by Das et al. [18], another high speed feature extraction module using feature sketches is presented by Pati et al. [52] and the implementation on virtex-II fpga was able to produce a maximum throughput of 3.32Gbps. Tong and Prasanna [65] proposed a method for anomaly detection on sketches. A generic architecture to accelerate sketches is proposed in this work and the analysis is performed on Count-Min (CM) sketch and K-ary sketch. The main design goal

of this work is to enhance the throughput by parallel and pipelined combinations of components. In online network anomaly detection, the CM sketch serves the purpose of online heavy hitter detection and K-ary sketch performs the online change detection. Keeping the width of the sketch $w=2^{16}$ and depth $d=5$, the implementation of the sketch acceleration on a virtex ultrascale xcvu440 platform leverages a throughput of 159Gbps for K-ary sketch and 155Gbps for CM sketch, which is lower only to the implementation in [57]. Lai et al. [33] propose a work on sketch-based DDoS detection on the NetFPGA SUME platform. In this work, the complex and time consuming shannon entropy computations are pre-computed and stored in lookup tables on the FPGA, which considerably reduce the latency and is able to achieve a throughput of 40Gbps.

Detection of heavy-hitters is very much relevant for DDoS, anomaly and intrusion detection. Tong and Prasanna [64] proposed an algorithm for an online heavy-hitter detection using CM sketch. With pipelining and parallel computing engines to accelerate the sketch, this architecture was able to achieve a throughput of 118Gbps on a virtex-7 platform. Heavy hitter detection on FPGA using a CM-CU sketch is proposed by Saavedra [55], where the advantage with CM-CU algorithm is that the memory requirement is small and only a 2-stage pipeline is required for the update operation. This is somewhat advantageous than [65] because the deep-pipeline requires complex hazard detection and data forwarding unit. When running at 400MHz, the CM-CU architecture of width $w=2^{14}$ and depth $d=4$ can achieve throughput of 128Gbps with no false positives on a Kintex-7 XC7K325T platform. Not many works on FPGA are targeted for Terabit Ethernet and Zazo et al. [76] presents a heavy hitter detection architecture on 100GbE links on a single FPGA. The detection architecture is based on Count Sketch (CS) and this work take advantage of the available 100G Ethernet subsystem on Xilinx Ultrascale devices. The main components in this design are the CS sketch and a priority queue (PQ), where the CS sketch populates the PQ and PQ keeps track of the most active flows. The memory requirement of the design is so low that both the CS sketch and PQ are able to fit in the on-chip BRAM. The implementation also achieves a frequency of 322MHz at 100Gbps line-rate. Most of the recent works like LOFT [58] for detecting overuse flows and Jaqen [40] for volumetric DDoS attack detection also underlines the capability of sketches on hardware to support processing at line-rate for Terabit Ethernet.

4 Machine learning-based Network Intrusion Detection

Instead of defining rules to detect intrusions, it is possible to use learning algorithms. Before considering FPGA implementations of such approaches in Sect. 4.2 through 4.4, we will first start by providing a brief introduction to machine learning for network intrusion detection in the next section.

4.1 Machine Learning for Network Intrusion Detection

In a machine learning-based network intrusion detection system, a machine learning algorithm is responsible for deciding what network traffic is malicious. Con-

cretely, the system is able to use specific features as input for a specific algorithm that as a result differentiates between benign traffic and attacks. While not necessarily new, this intrusion detection method remains challenging and is subject to continuous research. Initially, the detection algorithms mainly comprised traditional approaches such as Support Vector Machines (SVM) [32,6,5], decision trees (DT) [53,77] or simple neural networks (ANN) [45,41]. With the advent of Deep Learning (DL), larger and complexer architectures such as Convolutional Neural Networks (CNN), Long Short-Term Memory (LSTM) or autoencoders (AE) have become applicable for network intrusion detection. Some recent examples include [31,38,71]. Those learning algorithms are trained on datasets, usually publicly available, that present extracted traffic traces and/or features alongside their corresponding labels. Some frequently used examples include KDD99 [1], NSL-KDD [63], UNSW-NB15 [44] and CICIDS2017 [61]. While some datasets are outdated or consist of simulated traffic, they at least provide a common ground for comparison. Besides using published datasets, authors sometimes opt to instead use a custom dataset by combining other datasets [75] or using a custom testbed [25]. As these datasets are usually not available to other users, they can complicate comparison between different approaches.

The vast majority of research into machine learning for network intrusion detection however is limited to the design of software algorithms for improved detection of attacks through optimal feature and algorithm selection: Feature extraction and selection techniques are explored to obtain the features that best allow for distinguishing between normal and malicious traffic, while different machine learning models are investigated to optimally use the resulting features for decision making. Although this focus on software improvement is evidently important, it lacks consideration for the eventual implementation down the line. The resulting systems are often very resource intensive and have no innate support for feature extraction from raw network traffic. A complete NIDS should be able to interface with traffic in a network whilst consuming reasonable resources and returning feedback at an adequate rate. This is increasingly challenging as network bandwidths are ever increasing. Therefore, we investigate FPGA implementations of machine learning-based network intrusion detection systems as well as their capabilities and resource requirements. Sect. 4.2 considers traditional machine learning algorithms, while Sect. 4.3 lists current Deep Learning (DL) implementations on FPGA. Finally, Table 1 provides an overview of all discussed architectures with various metrics for comparison, which we will also discuss in Sect. 4.4.

4.2 Traditional machine learning algorithms on FPGA

When it comes to traditional machine learning algorithms, it is possible to differentiate between (shallow) neural network implementations and other implementations. While neural networks use calculations that are very similar to the calculations used in DL networks, other algorithms such as k-means clustering incorporate a different way to turn the provided input into a classification. Before moving on to neural networks, we will therefore first consider other approaches.

One of the earliest machine-learning based network intrusion detection systems on FPGA is provided by Das *et al.* [18] in 2008. They introduce a new feature extraction module (FEM) to summarize network behaviour, the output of which is fed to a *Principal Component Analysis* (PCA) to detect anomalies. The FEM first extracts relevant information from the traffic flow by updating *Feature Sketches* that track various properties. Retrieving values from those sketches produces the features that can be processed in the PCA. Anomalies are then finally detected by considering the deviation from the mean of the normal data set, taking into account both the most and the least significant principal components. The authors report a Detection Rate (also known as recall) of 99% with a False Alarm Rate of 1.95% on samples from the KDDCup1999 dataset. As they manage to combine this with a throughput of 23.76Gbps and a latency of 0.5 μ s, the resulting system is both accurate and fast.

Maciel *et al.* [8] use a modified version of the k-means clustering algorithm, namely k-modes, that can deal with categorical data of the NSL-KDD dataset. For comparison, they implemented integer and floating-point alternatives for k-means as well, running the three different versions on different features (counters, percentages or categorical features). All three approaches obtain significant improvements over their CPU counterparts, both in number of clock cycles used as well as in energy consumption. Moreover, of the three approaches, the *k-modes* algorithm on categorical data clearly outperforms the integer and floating-point alternatives. The authors however do not report the actual learning performance in terms of accuracy, or any other alternative metric.

Rather than k-means, Gordon *et al.* [25] use the k-nearest neighbours algorithm (KNN) to classify samples of a custom dataset on an FPGA. They propose a new sorting algorithm K-Min Sort to use during classification, which reduces resource overhead when compared to alternatives such as *Bubblesort*. For a training set size of 50%, they obtain an accuracy of 95% and a latency of 3.913ms while using a limited amount of hardware.

Although alternative learning methods such as k-means and KNN are used in some amount, the majority of learning architectures comprises neural networks: Structures consisting of a least one hidden layer with neurons to process input data. While software solutions more often use large, multi-layer neural networks, most neural networks for network intrusion detection on FPGA only consist of one hidden layer.

Murovič *et al.* set out to train and synthesize hardwired Binary Neural Networks (BNN) [46] for real-time and high-speed applications on FPGA. After training a 1-layer BNN of 100 neurons on UNSW-NB15 and obtaining an accuracy of 90.74%, they built a parallel Verilog implementation in hardware that has a latency of only 19.6ns. In [47] they propose a method to more efficiently implement BNN architectures in hardware. They replace the XNOR-popcount operation of binarized neurons with popcount operations preceded by precalculated masking. As a result, they reduce the hardware requirements for BNN layer computations. They also include a tool to map software BNNs to hardware.

When reaccelerating their previous 1-layer network using this novel approach, they report an improvement of 38.1% when compared to their previous results. While their latency has slightly increased, the power consumption is further reduced to 0.72W. In [48] they once again propose improved hardware acceleration methods, and they now train models for both NSL-KDD and UNSW-NB15. For their hardware, they introduce a ripple architecture based on the difference between neighbouring neurons in a layer. Moreover, as the hamming distance travelled when passing through a layer is correlated to the total number of summed bits, the authors use a genetic algorithm to minimize this distance. Using this approach, they manage to once improve upon their previous work. While the accuracy on NSL-KDD is rather limited at 77.77%, the hardware outperforms other work such as [8,28]. They also achieve very high throughputs of 256Gbps for UNSW-NB15 and 288Gbps for CICIDS2017 when considering packets of 576 bytes, which proves their architecture is very suited for high-speed applications.

Iannou *et al.* [28] implement a reconfigurable neural network on FPGA, that allows for updating coefficients at runtime. After selecting 29 input features from NSL-KDD, one-hot encoding them as necessary, they devise a model with 110 inputs, 29 hidden neurons and 2 outputs. They implement the model on a Xilinx Zynq Z-7020 FPGA using High-Level Synthesis (HLS), utilizing the included ARM to control the accelerator. Once the model is trained to reach an accuracy of 80.52%, they report a latency of 9.02ms for the implemented accelerator at a clock frequency of 76MHz.

Ngo *et al.* [50] design a decision tree and a neural network architecture on FPGA to classify NSL-KDD samples. They extract six features from input packets, and feed these features to the specific classifiers. While the neural network is very limited in size, using 6 inputs, two layers of 2 neurons and an output layer, it also features hardware for updating the weights using backpropagation. As the classifiers use 32-bit floating point precision arithmetic, the resource consumption is considerable. For packets of at least 512 bytes, the system manages a throughput of about 9.5Gbps, which quickly collapses as the packet size decreases in the case of the neural network. However, the decision tree retains a throughput of near line-speed even for 64-byte packets. In [49], the same authors propose network intrusion engines to secure Software Defined Networking forwarding devices, implementing a neural network in an FPGA after training it on a GPU. A controller presents 10 features to a neural network with 10 input units, one hidden layer with 4 neurons and an output layer to perform classification. The model reaches a throughput of 4.84Gbps with a recall of 99.01% on a custom dataset.

Somewhat different from the other research, Tran *et al.* [66] decide to split the machine learning challenge in two separate parts: They implement a feature extraction solution on an FPGA, but use a Graphical Processing Unit (GPU) to then perform neural network-based anomaly detection. On the FPGA, a *packet decoder* provides headers to the FEM, from which 10 features are extracted and normalized. An intermediate CPU facilitates the transfer of normalized data from FPGA to GPU. Although the utilized FPGA supports up to 10Gbps,

the bandwidth is limited to 200Mbps due to hashing collisions when storing statistical information in memory. The authors report 80.42% accuracy on a custom dataset.

4.3 Deep Learning on FPGA

Contrary to traditional learning methods and shallow neural networks, DL incorporates neural networks with various layers to perform more complex computations. While this increase in complexity has more potential with regards to classification performance, it comes paired with an increased cost to FPGA resources: Larger data structures and an increased number of computations will have to be dealt with to implement DL on FPGA.

Alrawashdeh *et al.* [7] accelerate the training and inference of a Deep Belief Network (DBN) on FPGA. Their design is not only able to detect intrusions in the NSL-KDD dataset, but also to (re)train the model while still online. The 3-layer DBN reuses the same hardware for each Restricted Boltzmann Machine (RBM) layer to limit hardware usage. Moreover, they use dynamic fixed-point arithmetic and limit the bit width of the second and third RBMs to 8 bits while maintaining 16 bits for the first layer. This results in a 30% resource usage improvement in hardware over using just 16-bit fixed point arithmetic. In addition, using an approximation function to replace the sigmoid activation function constitutes another optimization. They then evaluate, among others, on the NSL-KDD dataset, obtaining an accuracy of 94.66%, about 1.5% worse than the corresponding software implementation.

Umuroglu *et al.* [67] accelerate Deep Neural Networks (DNN) on an FPGA through their *LogicNets* methodology. This methodology is based on representing neurons in a DNN as truth tables on an FPGA, and allows for training the DNNs in PyTorch after which they can be translated to hardware. Among potential evaluation scenarios, they select network intrusion detection with the UNSW-NB15 dataset at very high bandwidths. They report an accuracy of 91.30% and a latency of 10.5ns when accelerating a 5-layer DNN with 593x256x128x128 neurons.

Le Jeune *et al.* [34] use two frameworks to quantize a convolutional neural network (CNN) and translate it to HLS. After using quantization aware training in Brevitas [51] to quantize their model's weights and activations to 2 bits, FINN [68,13] provides all the required steps to turn the resulting software model to hardware. Training on the CICIDS2017 dataset, they obtained an accuracy of 99.41%, with their reported hardware implementation obtaining a throughput of 90Mbps. While their throughput is significantly smaller than that of related work, the corresponding model complexity and classification performance is considerably higher than that of any other related work.

4.4 Discussion

The overview in Table 1 shows that FPGA implementations of machine learning algorithms for network intrusion detection vary greatly. Some implementations

Table 1. Overview of different FPGA implementations of machine learning algorithms for network intrusion detection. The columns feature, from left to right, the reference, the year of publishing, the used algorithm, the used dataset, the achieved accuracy, the supported network bandwidth, the model latency, the number of Lookup Tables, the number of registers, the power and the clock frequency.

Ref	Year	Algo	Dataset	Acc(%)	BW	Latency	LUTs	FF	P(W)	F(MHz)
[18]	2008	PCA	KDD99		23.76Gbps	0.5 μ s				93
[66]	2017	ANN	Custom	80.42	200Mbps		58561	71860		102
[7]	2017	DBN	NSL-KDD	94.3		8 μ s	121127	169290		
[50]	2019	ANN	NSL-KDD	87.30	9.58Gbps	12ms	107036	117078		104
[50]	2019	DT	NSL-KDD	95.10	9.58Gbps	4ms	67552	74059		100
[46]	2019	ANN	UNSW-NB15	90.74		19.6ns	51353	0	1.57	
[28]	2019	ANN	NSL-KDD	80.52		9.0ms	26463	56478		76
[8]	2020	k-means	NSL-KDD			1.2ms		31864	1.50	50
[49]	2020	ANN	Custom		4.84Gbps		99890	122085	11.00	100
[47]	2020	ANN	UNSW-NB15	90.74		23.5ns	26879	0	0.72	43
[67]	2020	DNN	UNSW-NB15	91.3		10.5ns	15949	1274		471
[48]	2021	ANN	NSL-KDD	77.77	288Gbps	16ns	26879	0	0.57	
[48]	2021	ANN	UNSW-NB15	92.04	256Gbps	19ns	26879	0	0.62	
[25]	2021	KNN	Custom	95		3.9ms	3663	3886		
[34]	2021	CNN	CICIDS2017	99.41	90Mbps		26615	27450		100

reach bandwidths of over 250Gbps, some occupy a limited area, and some retain a very high accuracy. In the ideal NIDS, all these factors should be present, but the current situation is that trade-offs have to be made: It is possible to reach blazing speeds using techniques such as those demonstrated by Murovič *et al.* [48] or in LogicNets [67], but this severely limits the computational size of the models, both in number of neurons and layers as well as in arithmetic. Binary neural networks are the pinnacle of such an approach. On the other hand, more expensive algorithms such as the DBN [7] or CNN [34] increase the accuracy, and therefore the reliability of the resulting NIDS. This is also essential, as inaccurate NIDSs will just flood a system administrator with false alarms or not detect actual attacks. Future machine learning-based NIDS implementations on FPGA will have to somehow compromise between these seemingly opposite goals of resource efficiency and accuracy.

5 Main takeaways

In this section, we compare the properties of the techniques covered in this paper as well as the limitations and open challenges in the state of the art.

5.1 Properties of the compared NIDS approaches

We will now compare the capabilities of pattern matching, large flow detection and machine learning through four key properties: detection capabilities,

expert knowledge, detection speed and hardware efficiency. To help practitioners, Table 2 summarizes these properties and their effect on the three discussed approaches.

Detection capabilities The detection capabilities of an NIDS comprise both the range of detectable attacks, as well as the detection accuracy for those attacks. For pattern matching-based detection this is quite clear: They detect all attacks that are effectively described in their ruleset. Traffic either violates a rule, or it does not. Attacks that however are not incorporated in the ruleset will not trigger any alarm and remain unnoticed. Flow measurement approaches are more dynamic than pattern matching techniques, but probabilistic measurement could cause false positives and this in fact would reduce the detection accuracy. Machine learning systems, in contrast, can potentially detect a wide range of attacks. Not only can they detect whatever is present in a training dataset, they also have the potential to detect attacks that are unknown if those attacks deviate from the trained model. The downside to machine learning is that the resulting systems often produce myriad errors, both in the form of unseen attacks as well as false alarms. Such false alarms can also be dangerous, as they might form a distraction for actual attacks.

Expert knowledge Different approaches to NIDS also result in different requirements of expert knowledge. Pattern matching-based approaches rely on rules, which have to be defined by experts. Without those rules, the system is useless. Moreover, whenever a new attack is discovered, a new rule must be made that effectively detects the attack. Machine learning on the other hand is based on a dataset, which the training algorithm uses to create its own rules internally. This removes the need for experts defining the precise characteristics of an attack. Updating a machine learning model can be done through updating its weights, for example by retraining. Flow measurement-based approaches fall somewhat in the middle, where the rules not necessarily have to be defined by the experts, but setting the parameters such as the threshold bandwidth, measurement period, and memory sizes would require high level of expertise.

Detection speed Attack detection does not only need to be accurate, it also needs to be fast. Fast detection not only facilitates swift response, it also helps in processing larger amounts of data. Generally, pattern-matching approaches can be very quick, with sub-microsecond latencies possible. However, the speed is tied to the size of the ruleset: As a system wants to check against more rules, it becomes harder to maintain speed. This also goes for the throughput, which can reach up to 400 Gbps for limited rulesets. However, in probabilistic architectures the size of the ruleset does not affect the detection speed. In machine learning, the speed is tied to the model size rather than the number of rules. As model types and sizes vary wildly, so does the detection speed, with measurements both in the millisecond and the nanosecond range (see Tab. 1). While extremely fast

models are certainly possible, these are often very limited in size, bit-width and accuracy. The most accurate models are therefore usually significantly slower and only support a limited bandwidth. The speed of online flow measurement-based systems is largely dependant on the speed of the measurement unit. Keeping the measurement unit within the on-chip memory of the FPGA reduces the latency to a minimum. However, with larger measurement period and higher memory requirement, offloading the storage to external memory significantly reduces the detection speed and even causes the monitoring to be performed offline.

Hardware efficiency The trade-offs that present themselves while considering detection speed, are similarly present when regarding hardware resource consumption. For pattern matching-based approaches, an increase in ruleset size implies an increase in resource consumption. In machine learning, the resources used depend strongly on the model used, considering the model size and the computational complexity. Once again, more accurate models will usually require more resources. It is however also possible to design models that are both accurate and (relatively) cheap in resource usage, but that might result in an increased latency instead. Similarly, the resource/memory requirement for measurement-based approaches is dependant on parameters like measurement period, traffic-rate, and accuracy.

Table 2. Summary of the most important properties of all three approaches

	Pattern matching	Flow measurement	Machine learning
Detection capabilities			
• Coverage	++	+	+++
• Accuracy	+++	++	+
Expert knowledge			
• Ease of maintenance	+	++	+++
• Ease of design	+++	++	+
Detection speed	+++	++	+
Hardware efficiency	+++	++	+

5.2 Limitations and challenges of state-of-the-art NIDSs

Encrypted traffic In the previous sections, different approaches to implement (parts of) IDS on FPGAs are discussed. One important remark to make is that, regardless of the approach, it is assumed that all network data are available to the IDS in plaintext. When connections use cryptography to encrypt the payload of network frames, pattern matching of this payload is infeasible. Depending on what type of encryption is used (e.g. IPSec in transport or tunnel mode or the new HTTP/3 (with QUIC⁷)) a number of protocol fields, or even entire protocols headers, are no longer available for inspection. With new secure Internet

⁷ <https://datatracker.ietf.org/doc/html/rfc9000>

Architectures like SCION [11], the packet headers of the available metadata also differ from what security experts are used to.

External components Most of the approaches that are mentioned earlier require a substantial amount of resources, and mainly memory resources. Going off-chip to external memory would cost too much time to achieve the required response times. It is therefore important to stay within the resources of one FPGA.

Update speed As network intrusion detection implies updates and changes at a quick pace, changes to the FPGA implementations also need to be fast. One way of speeding up the update process is through improving the FPGA toolchain. Another way is to use hierarchical or partial FPGA design and configuration.

6 Conclusion

Network intrusion detection systems on FPGA deal with the growing need for real-time intrusion detection in high-bandwidth networks. This paper discusses three important approaches, namely pattern matching, flow measurement and machine learning. An overview of both the employed techniques and the FPGA architectures is given, and the different methods are compared in terms of detection capabilities, expert knowledge, detection speed and hardware resources. Besides describing the clear potential of the different approaches, this paper also discusses the shortcomings and open challenges that need to be solved in order to be ready for the next generation of intrusion detection systems in high-speed networks.

Acknowledgements

This work is supported by CORNET and funded by VLAIO under grant number HBC.2018.0491. This work is also supported by the ESCALATE project, funded by FWO and SNSF (G0E0719N), and by Cybersecurity Initiative Flanders (VR20192203).

References

1. KDD Cup 1999 Data, <http://kdd.ics.uci.edu/databases/kddcup99/kddcup99.html>
2. AbuHmed, T., Mohaisen, A., Nyang, D.: A survey on deep packet inspection for intrusion detection systems. arXiv preprint arXiv:0803.0037 (2008)
3. Al-Dalky, R., Salah, K., Otrók, H., Al-Qutayri, M.: Accelerating snort NIDS using NetFPGA-based Bloom filter. In: 2014 International Wireless Communications and Mobile Computing Conference (IWCMC). IEEE (2014)

4. Al-Hisnawi, M., Ahmadi, M.: Deep packet inspection using cuckoo filter. In: 2017 NTICT. IEEE (2017)
5. Al-Qatf, M., Lasheng, Y., Al-Habib, M., Al-Sabahi, K.: Deep learning approach combining sparse autoencoder with SVM for network intrusion detection. IEEE Access **6** (2018)
6. Al-Yaseen, W.L., Othman, Z.A., Nazri, M.Z.A.: Multi-level hybrid support vector machine and extreme learning machine based on modified K-means for intrusion detection system. Expert Systems with Applications **67**, 296 – 303 (2017)
7. Alrawashdeh, K., Purdy, C.: Reducing calculation requirements in FPGA implementation of deep learning algorithms for online anomaly intrusion detection. In: 2017 IEEE National Aerospace and Electronics Conference (NAECON) (2017)
8. Andrade Maciel, L., Alcântara Souza, M., Cota de Freitas, H.: Reconfigurable FPGA-based K-Means/K-Modes architecture for network intrusion detection. IEEE Transactions on Circuits and Systems II: Express Briefs **67**(8) (2020)
9. Artan, N.S., Chao, H.J.: Multi-packet signature detection using prefix bloom filters. In: GLOBECOM'05. vol. 3. IEEE (2005)
10. Artan, N.S., Sinkar, K., Patel, J., Chao, H.J.: Aggregated bloom filters for intrusion detection and prevention hardware. In: IEEE GLOBECOM 2007-IEEE Global Telecommunications Conference. IEEE (2007)
11. Barrera, D., Chuath, L., Perrig, A., Reischuk, R.M., Szalachowski, P.: The scion internet architecture. Commun. ACM **60**(6) (May 2017)
12. Bloom, B.H.: Space/time trade-offs in hash coding with allowable errors. Communications of the ACM **13**(7) (1970)
13. Blott, M., Preußner, T.B., Fraser, N.J., Gambardella, G., O'brien, K., Umuroglu, Y., Leiser, M., Vissers, K.: FINN-R: An end-to-end deep-learning framework for fast exploration of quantized neural networks. ACM TRES **11**(3) (2018)
14. Češka, M., Havlena, V., Holík, L., Lengál, O., Vojnar, T.: Approximate reduction of finite automata for high-speed network intrusion detection. In: Beyer, D., Huisman, M. (eds.) Tools and Algorithms for the Construction and Analysis of Systems. pp. 155–175. Springer International Publishing, Cham (2018)
15. Češka, M., Havlena, V., Holík, L., Korenek, J., Lengál, O., Matoušek, D., Matoušek, J., Semric, J., Vojnar, T.: Deep packet inspection in FPGAs via approximate non-deterministic automata. In: 2019 IEEE 27th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM) (2019)
16. CISCO: CISCO IOS NetFlow Version 9. <http://www.cisco.com/c/en/us/products/ios-nx-os-software/netflow-version-9/index.html> (2015)
17. Cormode, G., Muthukrishnan, S.: An improved data stream summary: the count-min sketch and its applications. J. Algorithms **55**(1) (2005)
18. Das, A., Nguyen, D., Zambreno, J., Memik, G., Choudhary, A.: An FPGA-based network intrusion detection architecture. IEEE Transactions on Information Forensics and Security **3**(1) (2008)
19. Dharmapurikar, S., Krishnamurthy, P., Sproull, T., Lockwood, J.: Deep packet inspection using parallel bloom filters. In: 11th Symposium on High Performance Interconnects, 2003. Proceedings. IEEE (2003)
20. Dharmapurikar, S., Krishnamurthy, P., Taylor, D.E.: Longest prefix matching using bloom filters. In: Proceedings of the 2003 conference on Applications, technologies, architectures, and protocols for computer communications (2003)
21. Dharmapurikar, S., Lockwood, J.W.: Fast and scalable pattern matching for network intrusion detection systems. IEEE Journal on Selected Areas in communications **24**(10) (2006)

22. Dreger, H., Feldmann, A., Paxson, V., Sommer, R.: Operational experiences with high-volume network intrusion detection. In: ACM CCS (2004)
23. Fan, B., Andersen, D.G., Kaminsky, M., Mitzenmacher, M.D.: Cuckoo filter: Practically better than bloom. In: Proceedings of the 10th ACM International on Conference on emerging Networking Experiments and Technologies (2014)
24. Flajolet, P., Martin, G.N.: Probabilistic counting algorithms for data base applications. *Journal of computer and system sciences* **31**(2) (1985)
25. Gordon, H., Park, C., Tushir, B., Liu, Y., Dezfouli, B.: An efficient SDN architecture for smart home security accelerated by FPGA. In: 2021 IEEE International Symposium on Local and Metropolitan Area Networks (LANMAN) (2021)
26. Harwayne-Gidansky, J., Stefan, D., Dalal, I.: FPGA-based SoC for real-time network intrusion detection using counting Bloom filters. In: IEEE Southeastcon 2009. IEEE (2009)
27. Ho, T., Cho, S.J., Oh, S.R.: Parallel multiple pattern matching schemes based on cuckoo filter for deep packet inspection on graphics processing units. *IET Information Security* **12**(4) (2018)
28. Ioannou, L., Fahmy, S.A.: Network intrusion detection using neural networks on FPGA SoCs. In: 2019 29th International Conference on Field Programmable Logic and Applications (FPL) (2019)
29. Kang, J., Kim, T., Park, J.: FPGA-based real-time abnormal packet detector for critical industrial network. In: 2019 IEEE Symposium on Computers and Communications (ISCC) (2019)
30. Kefu, X., Deyu, Q., Zhengping, Q., Weiping, Z.: Fast dynamic pattern matching for deep packet inspection. In: 2008 IEEE ICNSC. IEEE (2008)
31. Khan, M.A.: Hcrnids: Hybrid convolutional recurrent neural network-based network intrusion detection system. *Processes* **9**(5) (2021)
32. Kim, D.S., Park, J.S.: Network-based intrusion detection with support vector machines. In: Kahng, H.K. (ed.) *Information Networking* (2003)
33. Lai, Y.K., Huang, P.Y., Lee, H.P., Tsai, C.L., Chang, C.S., Nguyen, M.H., Lin, Y.J., Liu, T.L., Chen, J.H.: Real-time DDoS attack detection using sketch-based entropy estimation on the NetFPGA SUME platform. In: 2020 Asia-Pacific Signal and Information Processing Association Annual Summit and Conference (APSIPA ASC). IEEE (2020)
34. Le Jeune, L., Goedemé, T., Mentens, N.: Towards real-time deep learning-based network intrusion detection on FPGA. In: ACNS Workshops (2021)
35. Li, C., Li, J., Yang, J., Lin, J.: A novel workload scheduling framework for intrusion detection system in NFV scenario. *Computers & Security* **106**, 102271 (2021)
36. Li, Y.Z.: Memory efficient parallel bloom filters for string matching. In: 2009 International Conference on Networks Security, Wireless Communications and Trusted Computing. vol. 1. IEEE (2009)
37. Lin, P.C., Lin, Y.D., Lai, Y.C., Zheng, Y.J., Lee, T.H.: Realizing a sub-linear time string-matching algorithm with a hardware accelerator using bloom filters. *IEEE transactions on very large scale integration (VLSI) systems* **17**(8) (2009)
38. Liu, L., Wang, P., Lin, J., Liu, L.: Intrusion detection of imbalanced network traffic based on machine learning and deep learning. *IEEE Access* **9** (2021)
39. Liu, Z., Manousis, A., Vorsanger, G., Sekar, V., Braverman, V.: One sketch to rule them all: Rethinking network flow monitoring with UnivMon. *Proc. ACM Special Interest Group Data Commun. (SIGCOMM)* (2016)
40. Liu, Z., Namkung, H., Nikolaidis, G., Lee, J., Kim, C., Jin, X., Braverman, V., Yu, M., Sekar, V.: Jaqen: A high-performance switch-native approach for detecting

- and mitigating volumetric DDoS attacks with programmable switches. In: 30th USENIX Security 21) (2021)
41. Lopez-Martin, M., Carro, B., Sanchez-Esguevillas, A., Lloret, J.: Shallow neural network with kernel approximation for prediction problems in highly demanding data networks. *Expert Systems with Applications* **124**, 196 – 208 (2019)
 42. Luinaud, T., Savaria, Y., Langlois, J.P.: An FPGA coarse grained intermediate fabric for regular expression search. In: GLSVLSI '17. ACM (2017)
 43. Morris, R.: Counting large numbers of events in small registers. *Commun. ACM* (1978)
 44. Moustafa, N., Slay, J.: UNSW-NB15: a comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set). In: 2015 Military Communications and Information Systems Conference (MilCIS) (2015)
 45. Mukkamala, S., Janoski, G., Sung, A.: Intrusion detection using neural networks and support vector machines. In: Proceedings of the 2002 International Joint Conference on Neural Networks. IJCNN'02 (Cat. No.02CH37290). vol. 2 (2002)
 46. Murovič, T., Trost, A.: Massively parallel combinational binary neural networks for edge processing. *Electrotechnical Review* **86**, 47–53 (01 2019)
 47. Murovič, T., Trost, A.: Resource-optimized combinational binary neural network circuits. *Microelectronics Journal* **97** (2020)
 48. Murovič, T., Trost, A.: Genetically optimized massively parallel binary neural networks for intrusion detection systems. *Computer Communications* **179** (2021)
 49. Ngo, D.M., Pham-Quoc, C., Thinh, T.N.: Heterogeneous hardware-based network intrusion detection system with multiple approaches for SDN. *Mobile Networks and Applications* **25**(3) (Jun 2020)
 50. Ngo, D.M., Tran-Thanh, B., Dang, T., Tran, T., Thinh, T.N., Pham-Quoc, C.: High-throughput machine learning approaches for network attacks detection on FPGA. In: Vinh, P.C., Rakib, A. (eds.) ICCASA and ICTCC 2017 (2019)
 51. Pappalardo, A.: Xilinx/brevitas, <https://doi.org/10.5281/zenodo.3333552>
 52. Pati, S., Narayanan, R., Memik, G., Choudhary, A., Zambreno, J.: Design and implementation of an FPGA architecture for high-speed network feature extraction. In: ICFPT. IEEE (2007)
 53. Pfahringer, B.: Winning the KDD99 classification cup: Bagged boosting. *SIGKDD Explor. Newsl.* **1**(2) (2000)
 54. Roh, J.h., Lee, S.k., Son, C.W., Hwang, C., Kang, J., Park, J.: Cyber security system with FPGA-based network intrusion detector for nuclear power plant. In: IECON 2020 The 46th Annual Conference of the IEEE Industrial Electronics Society. IEEE (2020)
 55. Saavedra, A., Hernández, C., Figueroa, M.: Heavy-hitter detection using a hardware sketch with the countmin-cu algorithm. In: 2018 21st Euromicro Conference on Digital System Design (DSD). IEEE (2018)
 56. Sateesan, A., Vliegen, J., Daemen, J., Mentens, N.: Novel bloom filter algorithms and architectures for ultra-high-speed network security applications. In: 2020 23rd Euromicro Conference on Digital System Design (DSD). IEEE (2020)
 57. Sateesan, A., Vliegen, J., Scherrer, S., Hsiao, H.C., Perrig, A., Mentens, N.: Speed records in network flow measurement on FPGA. In: Proceedings of the International Conference on Field-Programmable Logic (FPL) (2021)
 58. Scherrer, S., Wu, C.Y., Chiang, Y.H., Rothenberger, B., Asoni, D.E., Sateesan, A., Vliegen, J., Mentens, N., Hsiao, H.C., Perrig, A.: Low-rate overuse flow tracer (LOFT): An efficient and scalable algorithm for detecting overuse flows. arXiv preprint arXiv:2102.01397 (2021)

59. Schweller, R., Li, Z., Chen, Y., Gao, Y., Gupta, A., Zhang, Y., Dinda, P.A., Kao, M.Y., Memik, G.: Reversible sketches: enabling monitoring and analysis over high-speed data streams. *IEEE/ACM Transactions on Networking* **15**(5) (2007)
60. sFlow: Traffic Monitoring using sFlow. <http://www.sflow.org/sFlowOverview.pdf> (2003)
61. Sharafaldin, I., Habibi Lashkari, A., Ghorbani, A.: (01 2018)
62. Song, H., Lockwood, J.W.: Multi-pattern signature matching for hardware network intrusion detection systems. In: *GLOBECOM'05*. vol. 3. IEEE (2005)
63. Tavallaee, M., Bagheri, E., Lu, W., Ghorbani, A.A.: A detailed analysis of the KDD CUP 99 data set. In: *2009 IEEE Symposium on Computational Intelligence for Security and Defense Applications* (2009)
64. Tong, D., Prasanna, V.: High throughput sketch based online heavy hitter detection on FPGA. *ACM SIGARCH Computer Architecture News* **43**(4) (2016)
65. Tong, D., Prasanna, V.K.: Sketch acceleration on FPGA and its applications in network anomaly detection. *IEEE TPDS* **29**(4) (2017)
66. Tran, C., Vo, T.N., Thinh, T.N.: HA-IDS: A heterogeneous anomaly-based intrusion detection system. In: *NAFOSTED NICS 2017* (2017)
67. Umuroglu, Y., Akhauri, Y., Fraser, N.J., Blott, M.: LogicNets: Co-designed neural networks and circuits for extreme-throughput applications. In: *FPL 2020* (2020)
68. Umuroglu, Y., Fraser, N.J., Gambardella, G., Blott, M., Leong, P., Jahre, M., Vissers, K.: FINN: A framework for fast, scalable binarized neural network inference. In: *Proceedings of the 2017 ACM/SIGDA FPGA*. ACM (2017)
69. Wada, T., Matsumura, N., Nakano, K., Ito, Y.: Efficient byte stream pattern test using bloom filter with rolling hash functions on the FPGA. In: *2018 Sixth CANDAR*. IEEE (2018)
70. Wang, X., Hong, Y., Chang, H., Park, K., Langdale, G., Hu, J., Zhu, H.: Hyperscan: a fast multi-pattern regex matcher for modern CPUs. In: *USENIX NSDI* (2019)
71. Wang, Z., Zeng, Y., Liu, Y., Li, D.: Deep belief network integrating improved kernel-based extreme learning machine for network intrusion detection. *IEEE Access* **9** (2021)
72. Wellem, T., Lai, Y.K., Huang, C.Y., Chung, W.Y.: A hardware-accelerated infrastructure for flexible sketch-based network traffic monitoring. In: *IEEE 17th HPSR*. IEEE (2016)
73. Yang, T., Xu, J., Liu, X., Liu, P., Wang, L., Bi, J., Li, X.: A generic technique for sketches to adapt to different counting ranges. In: *IEEE INFOCOM* (2019)
74. Yang et al., T.: Elastic sketch: Adaptive and fast network-wide measurements. *Proc. ACM Special Interest Group Data Commun. (SIGCOMM)* (2018)
75. Yu, Y., Long, J., Cai, Z.: Session-based network intrusion detection using a deep learning architecture. In: Torra, V., Narukawa, Y., Honda, A., Inoue, S. (eds.) *Modeling Decisions for Artificial Intelligence* (2017)
76. Zazo, J.F., Lopez-Buedo, S., Ruiz, M., Sutter, G.: A single-FPGA architecture for detecting heavy hitters in 100 Gbit/s ethernet links. In: *2017 International Conference on ReConfigurable Computing and FPGAs (ReConFig)*. IEEE (2017)
77. Zhang, J., Zulkernine, M., Haque, A.: Random-forests-based network intrusion detection systems. *IEEE Transactions on Systems, Man, and Cybernetics, Part C (Applications and Reviews)* **38**(5) (2008)
78. Zhao, Z., Sadok, H., Atre, N., Hoe, J.C., Sekar, V., Sherry, J.: Achieving 100Gbps intrusion prevention on a single server. In: *14th USENIX OSDI20* (2020)
79. Zhou, Y., Zhang, Y., Ma, C., Chen, S., Odegbile, O.O.: Generalized sketch families for network traffic measurement. *POMACS* **3**(3) (2019)